

PONTIFICIA UNIVERSIDAD Católica DEL ECUADOR

Facultad de Ingeniería

Ingeniería en Sistemas y Computación

PROYECTO P1.3

Computación CPU & GPU Paralela: procesamiento simultáneo de cadenas de ADN sobre ambas plataformas

Asignatura: Computación Paralela, Distribuida y en la Nube

Docente: Ing. JW Condor

Autor: Alex Vivanco

Fecha de exposición: 22 de septiembre de 2026

1. Resumen

Este informe presenta una aplicación en Python que procesa el archivo FASTA del genoma humano GRCh38.p14, de 3,11 GB, poniendo a trabajar la CPU multinúcleo y la GPU de forma simultánea sobre el mismo archivo. A diferencia del proyecto anterior, en el que ambas plataformas se medían por separado para compararlas, aquí colaboran: el archivo se divide en trozos y cada plataforma toma los que puede atender mientras la otra trabaja, de modo que el reparto no se fija de antemano sino que emerge de la velocidad que cada una demuestra durante la ejecución.

Sobre el genoma completo, la ejecución paralela en CPU con doce procesos tomó 9.323 segundos, la ejecución en GPU tomó 1.488 segundos y la ejecución conjunta con cuatro procesos de CPU tomó 2.870 segundos, con una superposición real de 2.73 segundos durante los cuales ambas plataformas estuvieron activas. Todas las configuraciones produjeron conteos idénticos, verificados sobre el histograma completo de 256 valores de byte y no únicamente sobre las cuatro bases.

La aplicación instrumenta además el trabajo a nivel de núcleo: registra el uso de cada uno de los doce procesadores lógicos de la CPU y, dentro de la tarjeta, la cantidad de datos que procesa cada uno de sus veinte multiprocesadores, medida por los propios hilos durante la ejecución del kernel.

2. Planteamiento del problema

El ADN se representa computacionalmente como una cadena de caracteres construida sobre un alfabeto de cuatro símbolos que corresponden a las bases nitrogenadas adenina, citosina, guanina y timina. El conteo de frecuencias sobre esa cadena es una operación algorítmicamente trivial, pero deja de serlo cuando el volumen alcanza varios gigabytes, porque el costo deja de estar en el cálculo y pasa a estar en el acceso a los datos.

Los proyectos anteriores abordaron el problema plataforma por plataforma: primero repartiendo el trabajo entre los núcleos de la CPU y después trasladándolo a la GPU para comparar ambos resultados. El presente proyecto plantea una pregunta distinta y más exigente: si ambas unidades de cómputo pueden trabajar al mismo tiempo sobre el mismo archivo y si esa colaboración produce un beneficio.

La dificultad no está en lanzar las dos ejecuciones a la vez, sino en repartir el trabajo entre unidades de velocidad muy distinta y no conocida de antemano. Un reparto fijo por porcentajes exige saber cuánto más rápida es una que la otra, dato que depende del tamaño del archivo, del estado de la cache del sistema y de la carga del equipo. Si el reparto se calibra mal, la unidad más rápida termina su parte y queda ociosa esperando a la otra, y el conjunto rinde peor que la rápida trabajando sola.

El archivo utilizado sigue el formato FASTA, en el que las líneas que comienzan con el símbolo mayor que corresponden a cabeceras descriptivas y deben excluirse del conteo. Las líneas de

secuencia contienen además la letra N para bases desconocidas y emplean minúsculas como marcado de regiones repetitivas, que deben contarse igual que las mayúsculas.

3. Objetivos

- Desarrollar una aplicación que procese un archivo FASTA de varios gigabytes empleando la CPU multinúcleo y la GPU de forma simultánea sobre el mismo conjunto de datos.
- Diseñar un mecanismo de reparto de carga que no dependa de conocer de antemano la velocidad relativa de cada plataforma.
- Demostrar de forma medible que ambas plataformas estuvieron activas al mismo tiempo y cuantificar esa superposición.
- Instrumentar el uso de recursos a nivel de núcleo: cada procesador lógico de la CPU y cada multiprocesador de la GPU.
- Medir la temperatura y el consumo de memoria de ambas plataformas durante el procesamiento.
- Analizar si la ejecución conjunta resulta ventajosa frente a cada plataforma por separado e identificar el factor que lo determina.

4. Marco conceptual

4.1 Componentes de un computador

Un computador se organiza alrededor de cuatro bloques funcionales. La unidad central de proceso ejecuta las instrucciones; la memoria principal almacena de forma volátil los datos y el código en uso; el almacenamiento secundario conserva la información de forma permanente; y los buses interconectan todo lo anterior. A estos se suman hoy los coprocesadores, entre ellos la unidad de procesamiento gráfico, que dispone de su propia memoria y se comunica con el resto del sistema a través del bus PCIe.

Esa separación de memorias es determinante para este proyecto. Los datos que la GPU ha de procesar no residen en su memoria: deben viajar desde la memoria principal hasta la memoria de la tarjeta antes de poder ser tratados, y ese trayecto tiene un costo que la CPU no paga.

4.2 Propósito de una CPU frente al de una GPU

La CPU esta diseñada para minimizar la latencia de tareas heterogéneas y con dependencias entre si. Dedicar una fracción considerable de su superficie a memorias cache, predicción de saltos y ejecución fuera de orden, con el propósito de que un hilo individual avance lo más rápido posible. Dispone de pocos núcleos, pero cada uno es complejo y autónomo.

La GPU persigue el objetivo contrario: maximizar el rendimiento agregado sobre tareas homogéneas y sin dependencias. Dedicar su superficie a unidades aritméticas y organiza los hilos en grupos de treinta y dos, llamados warps, que ejecutan la misma instrucción sobre datos distintos. La tarjeta empleada en este proyecto agrupa esas unidades en veinte

multiprocesadores capaces de mantener hasta mil quinientos treinta y seis hilos activos cada uno.

El conteo de frecuencias se ajusta al segundo modelo: el resultado de un carácter no condiciona el de ningún otro. Por ello la GPU resulta la plataforma natural para esta carga, mientras que la CPU aporta capacidad adicional solo en la medida en que existan recursos libres para ella.

4.3 Computador personal frente a servidor empresarial

Un computador personal se optimiza para la interactividad de un único usuario: prioriza el costo, el consumo y la respuesta inmediata, y acepta que sus componentes trabajen de forma intermitente. Un servidor empresarial se optimiza para el rendimiento sostenido y la disponibilidad continua: incorpora memoria con corrección de errores, fuentes y discos redundantes, múltiples zócalos de procesador y sistemas de refrigeración dimensionados para operar sin interrupción.

La diferencia es visible en las mediciones de este informe. El equipo empleado es un portátil, y su almacenamiento se convirtió en el factor limitante mucho antes que su capacidad de cálculo. Un servidor destinado a este tipo de carga resolvería ese límite mediante arreglos de discos en paralelo o mediante la residencia de los datos en memoria, no mediante más potencia de cómputo.

4.4 Supercomputadores

Un supercomputador es un sistema construido para obtener el máximo rendimiento agregado en problemas científicos de gran escala. Se compone de miles de nodos de cómputo interconectados por una red de latencia muy baja, en su mayoría equipados con aceleradores gráficos, y se acompaña de sistemas de archivos paralelos, refrigeración líquida e instalaciones eléctricas dedicadas. Su rendimiento se expresa en operaciones de punto flotante por segundo y los sistemas actuales de primer nivel superan la escala de exaflops.

Su propósito es abordar problemas que no admiten solución en un plazo razonable sobre equipos convencionales: simulación climática, dinámica molecular, diseño de fármacos, astrofísica, ensayos nucleares simulados y, de forma creciente, entrenamiento de modelos de inteligencia artificial. La genómica figura entre sus aplicaciones habituales, y el problema tratado en este informe es una versión reducida de esa clase de cargas: el mismo principio de repartir un volumen grande de datos entre múltiples unidades de cómputo, aplicado aquí a dos unidades dentro de una sola máquina en lugar de a miles de nodos.

5. Ambiente de pruebas

Todas las mediciones se realizaron sobre el mismo equipo y bajo las mismas condiciones. Las especificaciones son las siguientes:

- Procesador: Intel Core i5-13420H, 13.ª generación, arquitectura híbrida.
- Núcleos físicos: 8. Procesadores lógicos: 12.

- Tarjeta gráfica: NVIDIA GeForce RTX 4050 Laptop GPU, arquitectura Ada Lovelace, compute capability 8.9, 20 multiprocesadores de flujo, 6140 MB de memoria VRAM, tamaño de warp de 32 hilos, 1536 hilos y 100 KB de memoria compartida por multiprocesador.
- Memoria RAM: 32 GB. Almacenamiento: unidad de estado sólido NVMe.
- Sistema operativo: Windows 11.
- Lenguaje: Python 3.13, con numba 0.67 y numba-cuda 0.30 para la GPU, multiprocessing para la CPU, llvmlite para la inyección de ensamblador PTX, y psutil, pynvml, matplotlib y tkinter para el monitoreo y la interfaz.
- Archivo de entrada: GCF_000001405.40_GRCh38.p14_genomic.fna, de 3,11 GB, y recortes de 50, 200 y 500 MB derivados de el.

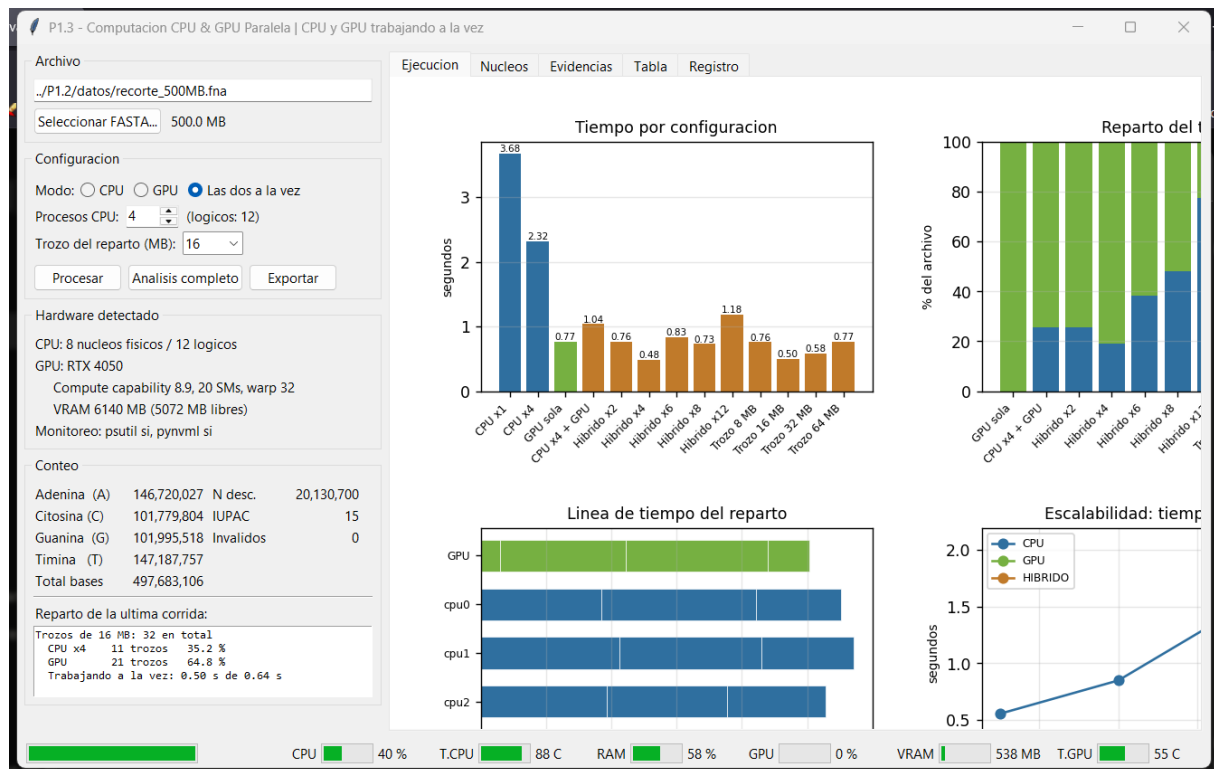


Figura 1. Aplicación desarrollada, con el hardware detectado y el reparto de la última ejecución en el panel izquierdo.

6. Diseño de la solución

6.1 Un mismo algoritmo sobre las dos plataformas

Ambos motores calculan un histograma completo de 256 posiciones, una por cada valor posible de byte, y derivan de el todos los conteos. La decisión es deliberada. Si la CPU empleara un atajo distinto al de la GPU, los tiempos dejarían de ser comparables, pero en este proyecto la consecuencia sería más grave: el mecanismo de reparto decide a quien entregar el siguiente trozo según la velocidad que cada plataforma demuestra, de modo que estaría repartiendo trabajos que no cuestan lo mismo y el equilibrio resultante no significaría nada.

6.2 El reparto por raciones adaptativas

El archivo se divide en trozos de tamaño fijo numerados de forma correlativa, y un único contador compartido indica cual es el siguiente trozo sin asignar. Cada trabajador, sea un proceso de CPU o el hilo que conduce la GPU, repite el mismo ciclo: reclama trabajo, lo procesa y vuelve a por más hasta que se agota.

Durante el desarrollo se comprobó que pedir un trozo por vez no basta. Con doce procesos de CPU y un hilo de GPU, los trece reclaman simultáneamente en el instante inicial y se reparten trece trozos antes de que ninguno haya demostrado su velocidad; si el archivo produce dieciséis trozos, el reparto queda decidido por el orden de llegada.

La solución adoptada consiste en que cada trabajador reclame una ración cuyo tamaño ajusta según su propia velocidad recién medida, tomando como objetivo la cantidad de trabajo que espera despachar en un cuarto de segundo. Todos comienzan pidiendo un solo trozo, en igualdad de condiciones. La GPU se estabiliza reclamando raciones amplias y los procesos de CPU se mantienen pidiendo de uno en uno. Ninguna proporción esta escrita en el programa: emerge de la medición, se readapta si las condiciones cambian y tiene el efecto adicional de entregar a la GPU tramos largos y contiguos, que es la forma en que mejor aprovecha la transferencia por el bus.

6.3 El descarte de cabeceras dentro de la tarjeta

Las cabeceras del formato FASTA no deben contarse. Eliminarlas en el equipo antes de enviar el lote a la tarjeta sería más sencillo, pero significaría que la CPU realiza parte del trabajo de la GPU, lo que distorsionaría tanto la comparación como el reparto de carga. Por ello el descarte ocurre dentro del dispositivo mediante tres kernels encadenados: el primero localiza el inicio de cada cabecera, el segundo la sobreescribe con saltos de línea, que el histograma ya ignora, y el tercero calcula el histograma acumulando primero en la memoria compartida de cada multiprocesador.

6.4 El arranque queda fuera de la medición

La compilación de los kernels requiere entre medio segundo y un segundo la primera vez, y la creación de doce procesos en Windows consume un tiempo comparable. En este proyecto excluir ese costo del cronometro no es solo una cuestión de limpieza metodológica: si el arranque quedara dentro de la medición, la plataforma que estuviera preparada antes reclamaría los primeros trozos por simple ventaja de salida, y el reparto observado no informaría sobre el rendimiento de ninguna de las dos. Por ese motivo el programa crea los procesos, los activa con tareas vacías y reserva la memoria de la tarjeta antes de iniciar el cronometro, y solo entonces da la salida a ambas plataformas de forma simultánea.

7. Implementación

La solución se estructuró en módulos independientes. El motor de CPU y el motor de GPU exponen la misma interfaz y devuelven el mismo diccionario de resultados; el motor híbrido los coordina; un módulo de monitoreo muestrea el uso de recursos en hilos de fondo; un banco de pruebas ejecuta los barridos; y la interfaz gráfica reúne todo lo anterior.

7.1 Código, librería e instrucción de cada nivel

La siguiente tabla identifica, para cada uno de los tres niveles de ejecución, el archivo que contiene el código correspondiente, la librería que habilita ese nivel y la instrucción concreta que pone a trabajar al hardware.

Criterio	CPU	GPU	Núcleos de GPU
Programa	motor_cpu.py. La función contar_paralelo reparte rangos de bytes entre procesos y contar_rango cuenta cada uno con numpy.bincount.	motor_gpu.py. Tres kernels encadenados: marcar las cabeceras, borrarlas dentro de la tarjeta y calcular el histograma de 256 bins.	motor_gpu.py, kernel k_histograma. Cada hilo acumula en la memoria compartida de su propio multiprocesador y anota en cual se ejecutó.
Librería	multiprocessing, de la biblioteca estándar, junto a numpy para el conteo vectorizado dentro de cada proceso.	numba.cuda, que compila a PTX los kernels escritos en Python y gestiona la memoria, los streams y los lanzamientos.	numba.extending.intrinsic junto a llvmlite.ir, que permiten inyectar ensamblador PTX dentro del kernel.
Instrucción	multiprocessing.Pool(processes=n) y pool.starmap_async(trabajo, tareas).	El decorador @cuda.jit sobre la función del kernel y el lanzamiento k_histograma[bloques, hilos, stream](datos, n, hist, por_sm).	La instrucción PTX mov.u32 \$0, %smid; leida mediante un intrinsic, y cuda.atomic.add(por_sm, smid(), vistos) para acumular su cuota.

7.2 La medición dentro de cada multiprocesador

Conocer el uso global de la tarjeta no permite saber como se distribuyó el trabajo entre sus multiprocesadores. Para obtener ese dato, el kernel del histograma consulta el registro especial que identifica el multiprocesador en el que se ejecuta cada hilo. Ese registro no es accesible desde Python, por lo que se recurre a un intrinsic de Numba que inyecta una instrucción de ensamblador PTX dentro del kernel compilado.

Cada hilo lleva la cuenta de los bytes que procesa en una variable local y la suma al contador de su multiprocesador una única vez, al terminar su bucle, y no en cada iteración. De este modo la medición no consume tiempo del trabajo que esta midiendo, y el reparto entre los veinte multiprocesadores queda registrado por los propios hilos en lugar de estimado.

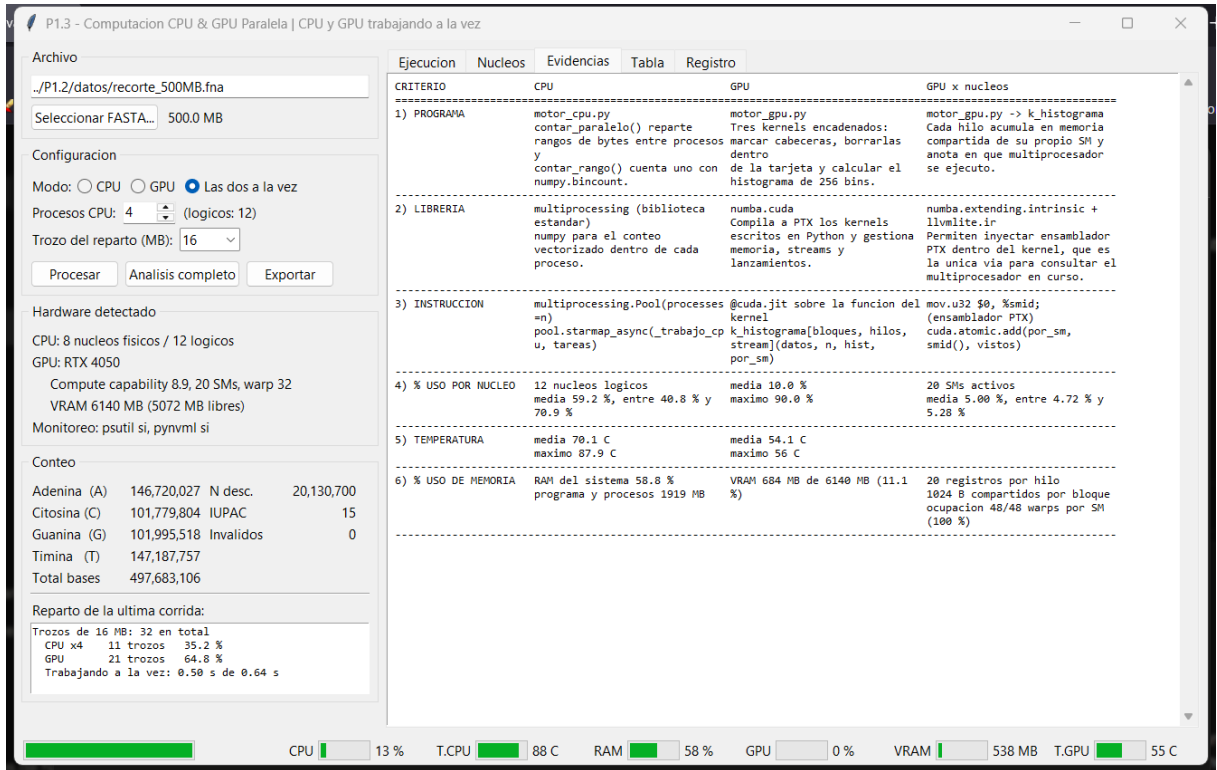


Figura 2. Pestaña de evidencias de la aplicación, que reúne en una sola vista el código, la librería, la instrucción y las mediciones de cada nivel.

7.3 Concurrencia y comportamiento en Windows

El procesamiento se ejecuta siempre en un hilo separado del de la interfaz. Dado que Tkinter no admite el acceso a sus componentes desde varios hilos, el hilo trabajador nunca modifica un componente de forma directa: deposita mensajes en una cola que la ventana vacía periódicamente. La GPU, por su parte, se conduce desde un hilo y no desde un proceso, porque el contexto de CUDA pertenece al proceso que lo creo y no puede compartirse con un hijo.

Un detalle del comportamiento de Windows condicionó la arquitectura de archivos. En este sistema multiprocessing emplea el método spawn, de modo que cada proceso hijo vuelve a importar el módulo principal. Durante el desarrollo del proyecto anterior se midió que la simple presencia de la importación del motor de GPU en el módulo principal elevaba el tiempo de una ejecución paralela sobre un archivo de 2 MB de 1,34 a 5,51 segundos, porque los procesos hijos cargaban la infraestructura de CUDA sin llegar a utilizarla. En el presente proyecto el perjuicio sería mayor, ya que ese retraso recaería sobre la mitad de CPU del motor justamente mientras compite por el reparto. Por ello el archivo de arranque contiene únicamente el lanzador y todos los componentes pesados se importan de forma diferida.

8. Resultados

8.1 Conteo de nucleótidos

El procesamiento del archivo completo arrojó los siguientes resultados, idénticos en todas las modalidades de ejecución:

Base	Ocurrencias	Porcentaje	Par complementario
Adenina (A)	923.117.203	29.43 %	Timina
Citosina (C)	642.552.917	20.48 %	Guanina
Guanina (G)	645.231.996	20.57 %	Citosina
Timina (T)	925.917.038	29.52 %	Adenina
Total de bases	3.136.819.154	100,00 %	-
N (desconocidas)	161.611.379	-	No aplica
Códigos IUPAC	103	-	No aplica

Los valores obtenidos admiten una validación independiente del programa. Dado que el ADN es una molécula de doble hebra en la que la adenina se empareja siempre con la timina y la citosina con la guanina, en un genoma completo ambas cantidades deben resultar prácticamente iguales. La diferencia observada entre adenina y timina es del 0.30 por ciento y la existente entre citosina y guanina del 0.42 por ciento, lo que confirma la correctitud del conteo por una vía biológica ajena al propio programa.

La verificación de correctitud se realizó sobre el histograma completo de 256 posiciones y no únicamente sobre las cuatro bases, y se repitió con distintas combinaciones de número de procesos y de tamaño de trozo, de manera que las uniones entre tramos consecutivos recayeran en posiciones diferentes del archivo en cada prueba. Todas las combinaciones produjeron histogramas idénticos.

8.2 Trabajo simultáneo de ambas plataformas

La aplicación registra el instante inicial y final de cada tramo junto con la identidad del trabajador que lo atendió, lo que permite construir la línea de tiempo de la ejecución. Esa representación constituye la evidencia directa de que ambas plataformas operaron de forma simultánea: si el programa se limitara a alternarlas, las bandas de CPU y de GPU no compartirían ninguna franja vertical.

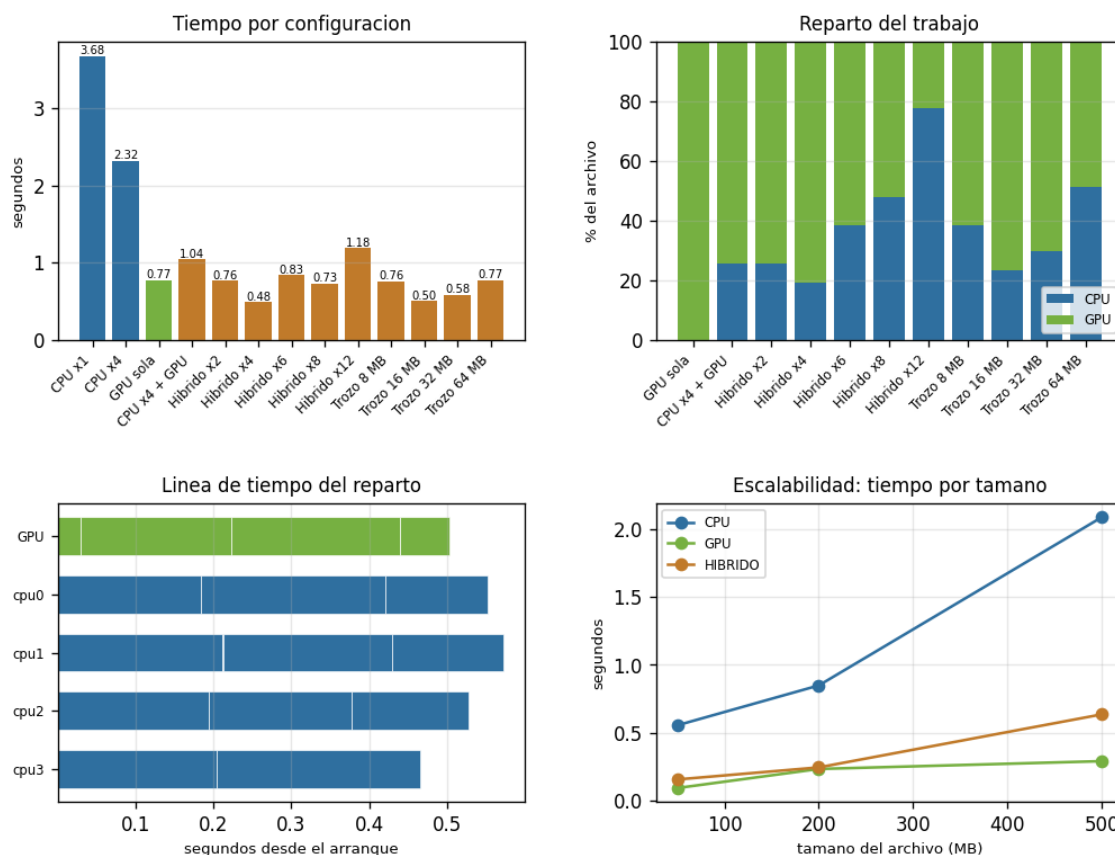


Figura 3. Tiempo por configuración, reparto del trabajo, línea de tiempo de la ejecución conjunta y escalabilidad.

Sobre el genoma completo, la configuración con cuatro procesos de CPU mantuvo ambas plataformas activas de forma simultánea durante 2.73 de los 2.870 segundos que duro la ejecución, es decir durante el 95 por ciento del tiempo total. El reparto resultante entregó el 21 por ciento del archivo a la CPU y el resto a la GPU, proporción que el programa no fija en ningún punto sino que deriva de la velocidad medida a cada una.

8.3 Desempeño comparativo

La tabla siguiente resume las mediciones sobre el genoma completo de 3,11 GB.

Configuración	Tiempo (s)	MB/s	Reparto CPU	Trabajo a la vez
CPU paralelo x12	9.323	341.6	-	-
GPU sola	1.488	2140.6	-	-
CPU x4 + GPU	2.870	1109.6	21 %	2.73 s
CPU x8 + GPU	3.326	957.7	32 %	3.01 s
CPU x12 + GPU	5.177	615.2	52 %	4.70 s

Sobre el recorte de 500 MB, que permite repetir muchas configuraciones en un tiempo razonable, los resultados fueron los siguientes:

Configuración	Tiempo (s)	MB/s	Speedup	Trabajo a la vez
CPU x1	3.675	136.1	1.00x	-
CPU x4	2.320	215.5	1.58x	-
GPU sola	0.765	653.6	4.80x	-
CPU x4 + GPU	1.038	481.6	3.54x	0.78 s

8.4 Cuántos procesos de CPU conviene sumar a la GPU

El número de procesos de CPU que acompañan a la tarjeta resultó ser el parámetro más influyente del sistema, y su efecto es contrario al que cabría esperar.

Configuración	Tiempo (s)	MB/s	Reparto CPU	Trabajo a la vez
Híbrido x2	0.761	656.7	25.6 %	0.62 s
Híbrido x4	0.484	1032.2	19.2 %	0.40 s
Híbrido x6	0.835	599.1	38.4 %	0.58 s
Híbrido x8	0.728	686.5	48.0 %	0.47 s
Híbrido x12	1.184	422.4	77.6 %	1.02 s

La configuración más rápida fue Híbrido x4, con 0.484 segundos, y la más lenta Híbrido x12, con 1.184 segundos, es decir 2.4 veces más tiempo para el mismo trabajo. A medida que aumenta el número de procesos, la CPU se lleva una fracción mayor del archivo, que pasa del 19.2 al 77.6 por ciento, y el tiempo total empeora. El mecanismo de reparto funciona correctamente en todos los casos, en el sentido de que ninguna plataforma permanece ociosa; lo que ocurre es que el trabajo entregado a la CPU resulta más costoso que el mismo trabajo entregado a la tarjeta. El análisis de la sección siguiente examina la causa.

8.5 Grano del reparto

El tamaño de los trozos en que se divide el archivo constituye la perilla de ajuste del balanceo de carga. Con trozos grandes hay pocas piezas que repartir y el resultado lo decide el orden de llegada; con trozos demasiado pequeños se paga muchas veces el costo fijo de posicionarse en el archivo.

Trozo	Tiempo (s)	MB/s	Reparto CPU	Trabajo a la vez
Trozo 8 MB	0.758	659.3	38.4 %	0.62 s
Trozo 16 MB	0.499	1001.0	23.2 %	0.39 s
Trozo 32 MB	0.581	861.1	29.6 %	0.43 s
Trozo 64 MB	0.767	652.1	51.2 %	0.30 s

El mejor resultado se obtuvo con trozo 16 mb, con 0.499 segundos. La diferencia entre el mejor y el peor grano es de 53.5 por ciento, sensiblemente menor que la que provoca el número de procesos, lo que confirma que el parámetro determinante no es como se trocea el archivo sino cuantos consumidores compiten por leerlo.

8.6 Uso por núcleo de ambas plataformas

Durante el análisis completo, los doce procesadores lógicos registraron un uso medio del 59.2 por ciento, con valores comprendidos entre el 40.8 y el 70.9 por ciento. La dispersión refleja la arquitectura híbrida del procesador, en la que conviven núcleos de rendimiento y núcleos de eficiencia con capacidades distintas.

Dentro de la tarjeta, los veinte multiprocesadores recibieron entre el 4.72 y el 5.28 por ciento de los bytes procesados, frente al 5,00 por ciento que correspondería a un reparto perfectamente uniforme. La desviación, inferior a tres décimas de punto porcentual, procede de la forma en que el planificador de la tarjeta distribuye los bloques entre los multiprocesadores y resulta despreciable a efectos prácticos.

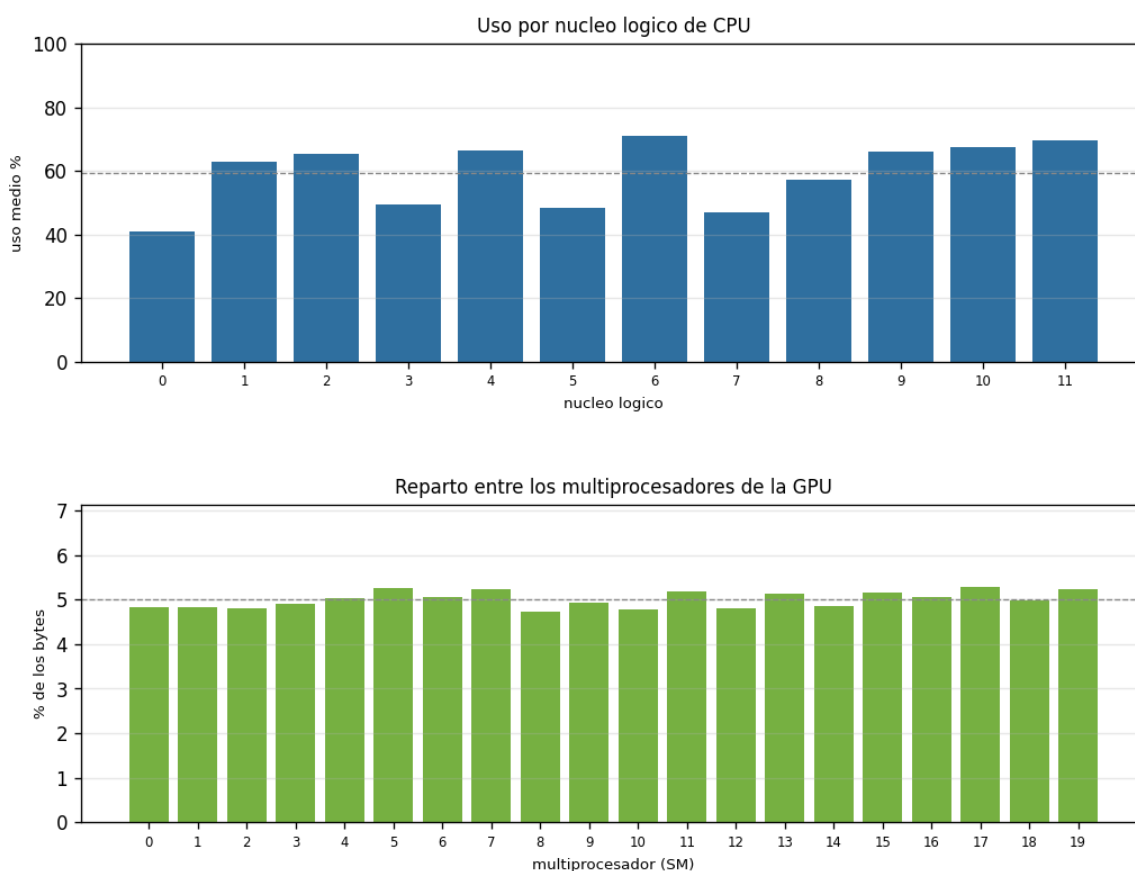


Figura 4. Uso medio de cada procesador lógico de la CPU y reparto de los datos entre los veinte multiprocesadores de la tarjeta.

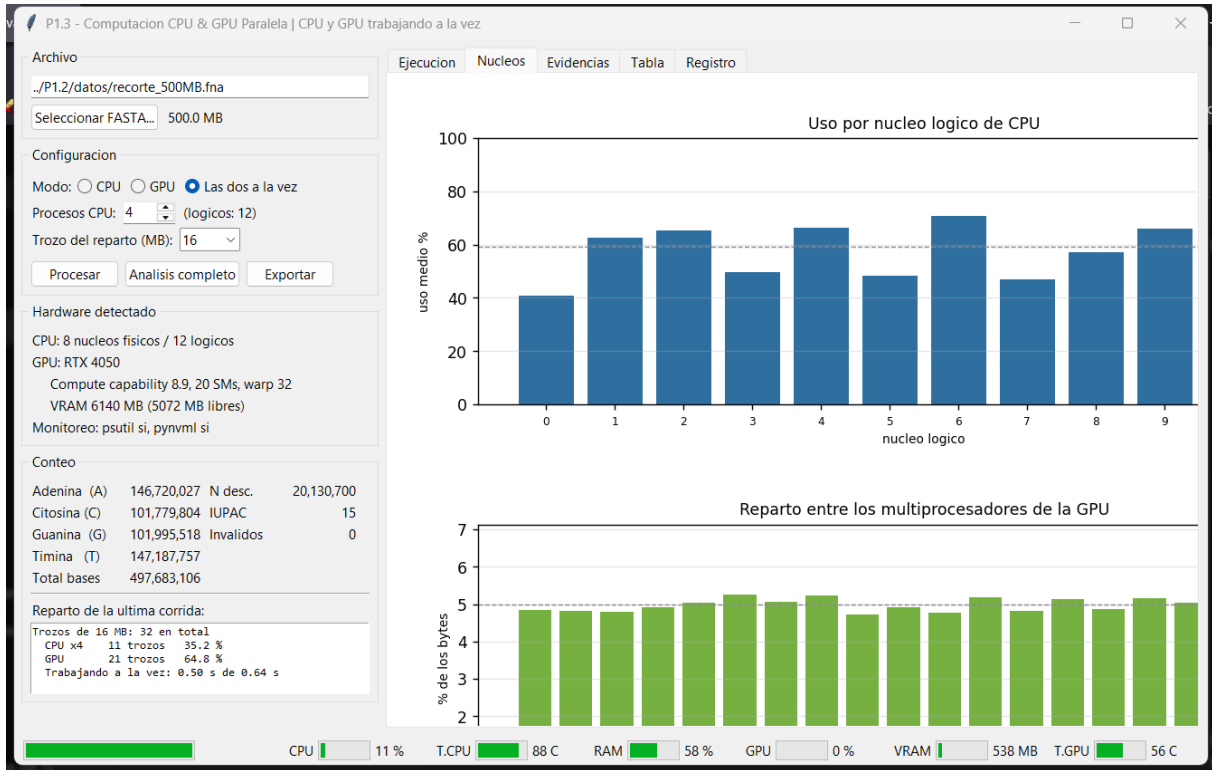


Figura 5. La misma información dentro de la aplicación, con las lecturas de recursos en vivo en la barra inferior.

8.7 Temperatura y uso de memoria

El monitoreo registro durante el análisis una temperatura media del procesador de 70.1 grados centígrados, con un máximo de 87.9, y una temperatura media de la tarjeta de 54.1 grados con un máximo de 56. La diferencia es coherente con el reparto observado y con el hecho de que la tarjeta despacha su parte del trabajo en una fracción del tiempo.

En cuanto a la memoria, el sistema alcanzó un 58.8 por ciento de ocupación y el conjunto del programa y sus procesos hijos llegó a 1.919 MB de memoria residente. La tarjeta ocupó 684 MB de sus 6.140 MB de VRAM. Se informa la memoria del programa y no la del sistema completo porque esta última está dominada por el resto de aplicaciones abiertas y ocultaría por completo el costo que corresponde a cada estrategia.

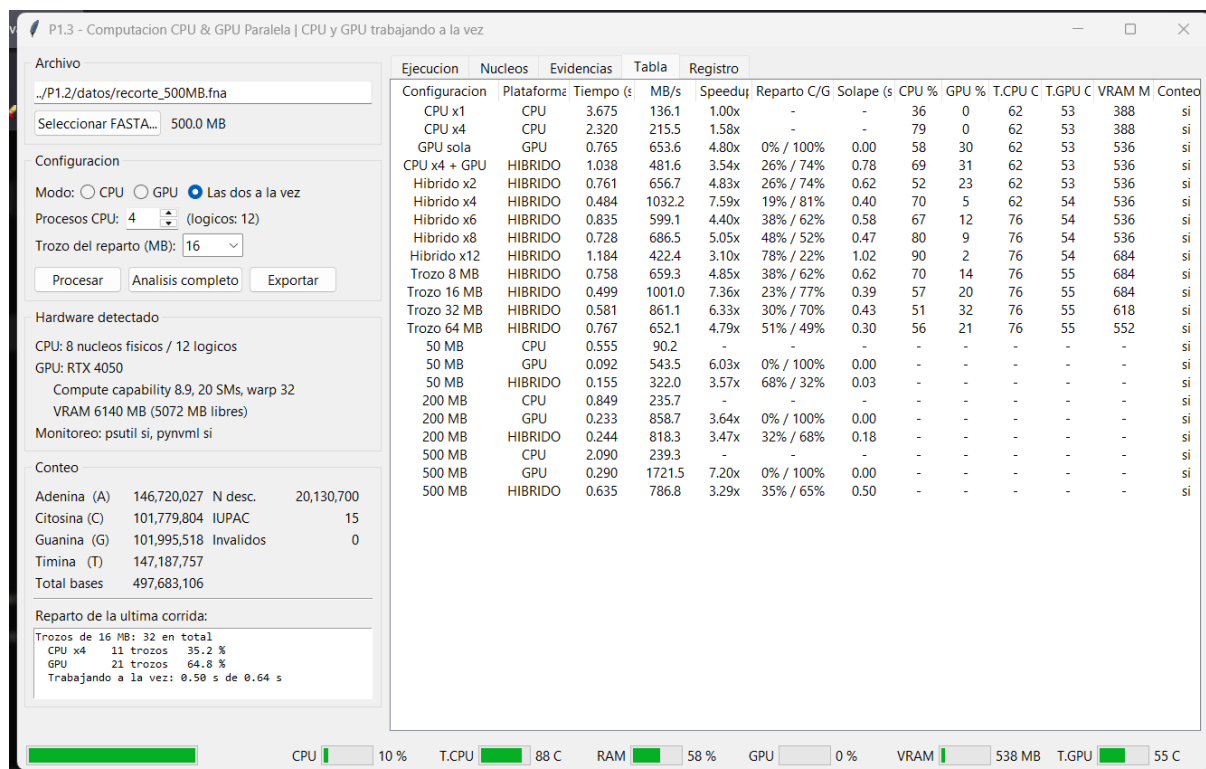
A nivel de multiprocesador, el kernel del histograma consume 20 registros por hilo y 1.024 bytes de memoria compartida por bloque. Con ese consumo la tarjeta admite 48 de los 48 warps que un multiprocesador puede mantener activos, es decir una ocupación del 100 por ciento, que es el valor máximo alcanzable.

8.8 Escalabilidad

El último barrido midió el comportamiento de las tres modalidades frente a volúmenes crecientes de datos.

Tamaño	CPU (s)	GPU sola (s)	Conjunta (s)	GPU frente a CPU
50 MB	0.555	0.092	0.155	6.03x
200 MB	0.849	0.233	0.244	3.64x
500 MB	2.090	0.290	0.635	7.20x

La ventaja de la tarjeta se mantiene en todo el rango medido y crece con el volumen, lo que confirma que su costo fijo de preparación se amortiza ya en los tamaños más pequeños de esta serie. La modalidad conjunta se sitúa de forma sistemática entre ambas, sin alcanzar en ningún tamaño a la tarjeta trabajando sola.



9. Análisis critico

9.1 La colaboración funciona pero no compensa

El objetivo central del proyecto se cumplió: ambas plataformas procesan el mismo archivo de forma simultánea, la superposición se mide en segundos y el reparto se equilibra por si solo sin que ninguna proporción este fijada en el programa. Sin embargo, el conjunto resulta más lento que la tarjeta trabajando sola, y cuantos más procesos de CPU se incorporan, peor es el resultado. Sobre el genoma completo la GPU sola tarda 1.488 segundos mientras que la ejecución conjunta con doce procesos tarda 5.177.

Conviene subrayar que esto no constituye un defecto del mecanismo de reparto. El reparto hace exactamente lo que debe: ninguna plataforma permanece ociosa y cada una recibe una cantidad de trabajo proporcional a la velocidad que demuestra. El problema es de otra naturaleza y se examina a continuación.

9.2 El cuello de botella no esta en el calculo

Contar bytes es una operación de coste computacional mínimo: una lectura de memoria y un incremento por cada carácter. En consecuencia, el factor que limita el rendimiento no es la capacidad de calculo de ninguna de las dos plataformas sino la velocidad a la que los datos llegan desde el almacenamiento. La GPU por si sola alcanza los 2140.6 MB/s sobre el genoma completo, cifra que corresponde al límite práctico de lectura del sistema.

Los procesos de CPU no aportan capacidad de lectura adicional; compiten por la que ya existe. Peor aun, restan turnos de acceso al único hilo lector que alimenta a la tarjeta, que es precisamente el que obtiene mayor rendimiento de cada byte recibido. De ahí la progresión observada: con dos procesos la perdida es moderada, mientras que con doce el conjunto rinde a menos de un tercio de lo que rendiría la tarjeta sola.

La conclusión general que se desprende es que la ejecución conjunta de dos unidades de cómputo solo resulta ventajosa cuando el factor limitante es el cómputo. Cuando el límite reside en el acceso a los datos, ningún reparto, por perfecto que sea, puede superar el ancho de banda del componente más lento de la cadena, y añadir consumidores de ese recurso escaso lo único que consigue es repartirlo peor.

9.3 El reparto dentro de la tarjeta

Frente al desequilibrio observado entre las dos plataformas, el reparto interno de la tarjeta resultó notablemente uniforme: los veinte multiprocesadores procesaron entre el 4.72 y el 5.28 por ciento de los datos. El contraste es ilustrativo, porque ambos repartos obedecen a mecanismos distintos. El reparto entre multiprocesadores lo realiza el planificador de la tarjeta sobre unidades idénticas y con acceso a la misma memoria, situación en la que un reparto uniforme es el resultado esperable. El reparto entre CPU y GPU, en cambio, se produce entre unidades de naturaleza distinta que compiten por un recurso externo y limitado.

9.4 Un error de borde en las uniones entre tramos

Durante la verificación se detectó que el conteo resultaba incorrecto en aproximadamente una de cada cinco ejecuciones, sin patron aparente. El origen estaba en el tratamiento del final de cada tramo por parte del motor de GPU: al cerrar el último lote llamaba a la función de lectura de línea sin comprobar previamente si la línea había quedado efectivamente cortada. Cuando un tramo terminaba justo en un salto de línea, esa llamada devolvía la línea siguiente completa, que pertenecía al tramo contiguo, y esos ochenta caracteres acababan contados dos veces.

El fallo permanecía oculto por dos razones. Las divisiones se producen cada dieciséis megabytes y las líneas miden ochenta y un bytes, de modo que la coincidencia es infrecuente; y además

solo se manifestaba cuando el tramo que terminaba en un salto correspondía a la GPU y no a la CPU, que si realizaba la comprobación. Como el reparto varia en cada ejecución, el error aparecía y desaparecía sin causa aparente.

El episodio deja una lección metodológica aplicable a cualquier sistema con reparto dinámico: una única ejecución correcta no demuestra nada, porque las uniones entre tramos recaen en posiciones distintas en cada una. Por ese motivo el programa de verificación ejecuta varias combinaciones de número de procesos y de tamaño de trozo, con el propósito expreso de desplazar esas uniones a lo largo del archivo.

10. Aplicaciones reales

El escenario planteado constituye una versión simplificada de problemas habituales en bioinformática. El alineamiento de secuencias contra un genoma de referencia, la detección de variantes genéticas, el calculo del contenido de guanina y citosina por regiones o el control de calidad de los datos producidos por un secuenciador operan sobre volúmenes comparables o superiores.

La ejecución conjunta sobre plataformas heterogéneas es hoy la norma en los sistemas de alto rendimiento. Los entornos de cómputo científico reparten el trabajo entre procesadores y aceleradores, y los marcos de inteligencia artificial distribuyen las operaciones entre ambos según su naturaleza. El mecanismo de reparto dinámico implementado en este proyecto es una versión reducida del que emplean esos sistemas.

El resultado obtenido tiene además valor práctico más alla del caso concreto. Antes de invertir esfuerzo en repartir una carga entre varias unidades de cómputo conviene determinar donde reside el factor limitante. Si se encuentra en el acceso a los datos, la solución no pasa por sumar unidades de calculo sino por mejorar el camino de los datos: almacenamiento más rápido, lectura en paralelo desde varios dispositivos o residencia de los datos en memoria.

11. Conclusiones

- Se implementó un motor que procesa un mismo archivo con la CPU multinúcleo y la GPU trabajando de forma simultánea. Sobre el genoma completo ambas plataformas permanecieron activas al mismo tiempo durante 2.73 de los 2.870 segundos de la ejecución.
- El reparto de carga se resuelve mediante raciones adaptativas: cada trabajador ajusta la cantidad de trabajo que reclama según su propia velocidad medida. Ninguna proporción entre plataformas esta fijada en el programa, y el reparto obtenido es una consecuencia medida y no una suposición.
- La ejecución conjunta resultó más lenta que la GPU trabajando sola: 2.870 segundos frente a 1.488 sobre el genoma completo en la mejor configuración conjunta. El factor determinante es que el problema esta limitado por el acceso a los datos y no por la capacidad de calculo.

- El número de procesos de CPU que acompañan a la tarjeta resultó el parámetro más influyente: entre la mejor y la peor configuración del barrido hay un factor de 2.4 para el mismo trabajo, porque todos ellos compiten por el mismo ancho de banda de lectura.
- La instrumentación a nivel de núcleo mostró un reparto interno de la tarjeta muy uniforme, entre el 4.72 y el 5.28 por ciento de los datos por multiprocesador, y una ocupación del 100 por ciento de los warps disponibles.
- La correctitud se verificó sobre el histograma completo de 256 valores de byte y con múltiples configuraciones de reparto, de modo que las uniones entre tramos recayeran en posiciones distintas. Todas las combinaciones produjeron conteos idénticos.
- La proporción obtenida entre bases complementarias, con diferencias inferiores al 0,31 por ciento entre adenina y timina y al 0,42 por ciento entre citosina y guanina, valida el resultado por una vía biológica independiente del programa.

12. Recomendaciones

- Evaluar la lectura del archivo mediante varios hilos de entrada y salida con profundidad de cola elevada, dado que las unidades NVMe admiten múltiples peticiones simultáneas y podrían superar el límite alcanzado por el acceso secuencial.
- Repetir el experimento con una carga de mayor costo computacional por byte, como la búsqueda de patrones o el cálculo de contenido de guanina y citosina por ventanas, para verificar la hipótesis de que la ejecución conjunta resulta ventajosa cuando el límite reside en el cómputo.
- Incorporar al mecanismo de reparto una fase inicial de calibración que mida ambas plataformas por separado antes de comenzar, de modo que las primeras raciones partan de una estimación más ajustada.
- Repetir cada configuración varias veces y reportar el promedio, con el fin de reducir el efecto de la cache del sistema operativo, cuya influencia sobre las cifras absolutas resultó considerable.
- Considerar la implementación del motor de CPU en un lenguaje compilado para establecer el límite superior real de esa plataforma y obtener una comparación más estricta frente a la tarjeta.

13. Referencias

- NVIDIA Corporation. CUDA C++ Programming Guide. Disponible en: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- NVIDIA Corporation. Parallel Thread Execution ISA. Disponible en: <https://docs.nvidia.com/cuda/parallel-thread-execution/>
- Anaconda Inc. Numba Documentation: CUDA Python. Disponible en: <https://numba.readthedocs.io/en/stable/cuda/index.html>
- Python Software Foundation. Documentación oficial del módulo multiprocessing. Disponible en: <https://docs.python.org/3/library/multiprocessing.html>

- National Center for Biotechnology Information. Genome Reference Consortium Human Build 38 (GRCh38). Disponible en: https://www.ncbi.nlm.nih.gov/assembly/GCF_000001405.40/
- Cornish-Bowden, A. Nomenclature for incompletely specified bases in nucleic acid sequences. Nucleic Acids Research, vol. 13, num. 9, 1985.
- TOP500.org. TOP500 List of the most powerful supercomputer systems. Disponible en: <https://www.top500.org/>
- NVIDIA Corporation. Especificaciones de la arquitectura Ada Lovelace y de la serie GeForce RTX 40. Disponible en: <https://www.nvidia.com/>